TransitPro

Team guide
Who builds what.

Modules, UML diagrams for the CA, screens, deadlines, coding rules and the answers to the lecturer's questions.

PROJECT LEAD
Fanyi Charllson

COURSE
Large System Environment

VERSION
1.0 · 5 November 2026

TEAM
5 members

1 The project in one page

TransitPro is a SaaS platform for intercity bus agencies in Cameroon. Agencies sign up, are verified by us (RCCM, NIU, transport licence), then run their whole business on it: ticket sales, buses, parcels, staff and money. Customers compare verified agencies, book seats and track parcels from their phone.

Five modules share **one central PostgreSQL database**. When something changes in one module, the others see it: a bus in maintenance stops selling seats, every ticket and parcel sale posts to Finance, a driver with an expired licence cannot be assigned to a trip.

PART	WHO USES IT	TECHNOLOGY
Server (`server/`)	Everything talks to it	Express 5, TypeScript, Prisma, PostgreSQL
Web dashboard (`web/`)	Agency office: counter agents, managers, HR, accountant, owner	React 19, Vite, Tailwind v4
Mobile app (`mobile/`)	Customers, plus drivers and couriers in staff mode	Expo SDK 54, Expo Router

Security choices (everyone must respect them)

- Login session in an **httpOnly cookie**: the token never reaches JavaScript, localStorage or AsyncStorage.
- **Role-based access** on every route: Customer, Counter agent, Driver, Courier, Branch manager, Fleet manager, HR officer, Accountant, Agency owner, Platform admin.
- Each agency only sees its own data. Customers can never become staff: staff accounts only come from an HR invite.
- Delivery codes and invite tokens are stored hashed. Public pages show no names or phone numbers.

Links

Repository	github.com/fanyicharllson/transitpro
UI design	claude.ai/artifact/1NiJFaNhTtXLuCb5VuzRaw · pages: Web dashboard, Customer app, Staff app
Coding rules	`AGENTS.md` at the root of the repo. Give it to your AI agent first.
API contract	`docs/API.md` : every endpoint, who can call it, which screen uses it
Database	`server/prisma/schema.prisma` : also the base of the shared class diagram

2 Who does what

Everyone owns one module: its **4 UML diagrams** for the CA and its **server code**. Then each person builds the screens listed below.

FC Fanyi Charllson — Project Lead

Module 3 · Parcel & Courier

SERVER

`modules/parcels`, `auth`, `agencies`, shared code, database

SCREENS

Web: layout, Dashboard, Register parcel, Parcel detail, Live tracking, Trips & booking

Use case: Sender, Receiver, Counter agent, Driver, Courier, Branch manager

Class: Parcel, ParcelEvent, ParcelClaim (+ Escrow, Employee, Branch, Trip)

Sequence: Courier delivers with the receiver's code, escrow released

Activity: Parcel journey with weight check and refusal branches

AT Adrien Tello — Mobile · customer app

Module 1 · Booking & Ticketing

SERVER

`modules/booking`

SCREENS

Mobile `app/(auth)` + `app/(customer)`: login, sign-up, home, search, results, agencies, agency profile, seat, pay, e-ticket, track parcel

Use case: Passenger, Counter agent, Driver, Payment provider

Class: Route, Trip, Ticket, Review (+ Bus, Payment, User)

Sequence: Passenger books a seat and pays with MoMo

Activity: Search → seat → pay → ticket ("seat taken", "payment failed")

NF Ngwese Fidel — Mobile · staff app

Module 2 · Fleet & Maintenance

SERVER

`modules/fleet` (buses, maintenance, fuel, breakdowns, live GPS)

SCREENS

Mobile `app/(staff)` + `app/invite`: invite, Courier Today, courier delivery (code entry), Driver Today, QR scanner

Use case: Fleet manager, Driver, Branch manager

Class: Bus, MaintenanceRecord, FuelLog (+ Trip, Employee, Expense)

Sequence: Breakdown → bus in maintenance → trips stop selling → cost to Finance

Activity: Bus maintenance cycle (due → in maintenance → active)

Joy

Module 4 · Human Resources

SERVER

`modules/hr` (staff, invites, licences, payroll)

SCREENS

Web: Staff (list, add staff = invite, licence alerts, payroll) and the Fleet page

Use case: HR officer, Agency owner, Staff member

Class: Employee, StaffInvite (+ User, Role, Branch)

Sequence: HR invites a courier → SMS link → sets password → staff mode

Activity: Assigning a driver to a trip (licence valid? active?)

Sweet

Module 5 · Finance & Reporting

SERVER

`modules/finance` (payments, escrow, expenses, cash reconciliation)

SCREENS

Web: Finance (payments, expenses, daily cash), dashboard numbers and revenue chart

Use case: Accountant, Branch manager, Courier, Payment provider

Class: Payment, Escrow, Expense, CashRemittance (+ Ticket, Parcel)

Sequence: MoMo payment confirmed by the provider webhook

Activity: Daily cash reconciliation (settled / short)

Shared class diagram (whole group, led by Charllson): all models of the central database and their relations. It proves the 5 modules share one database. In your own class diagram, draw your classes in full and other modules' classes with only their name and key fields. In your sequence diagram, show the call into the other module's service (e.g. `ParcelService → FinanceService`): that interconnection is what the lecturer grades.

3 Mobile: Adrien and Fidel

One Expo app, two experiences. After login, `GET /api/auth/me` returns the role. `CUSTOMER` opens the customer app; `DRIVER` or `COURIER` open the staff app. Customers never see staff screens or wording. Staff cannot sign up: their agency invites them, and the SMS link opens `app/invite/[token]` to set a password.

FOLDER	DESIGN SCREENS	OWNER
`app/(auth)/`	Log in, Create account (Google + email/password)	Adrien
`app/(customer)/(tabs)/`	Home, Trips, Parcels, Profile (bottom bar)	Adrien
`app/(customer)/search/`	Search · where to?, Search results	Adrien
`app/(customer)/agency/`	Verified agencies, Agency profile	Adrien
`app/(customer)/booking/`	Choose a seat, Pay, E-ticket	Adrien
`app/(staff)/(tabs)/`	Courier Today, Driver Today, Scan, Cash, Profile	Fidel
`app/(staff)/deliver/`	Courier delivery: receiver's code, photo, GPS	Fidel
`app/invite/`	Invite · set password	Fidel

Dependencies are not in the repo to save space. Mobile devs run `npm install` then `npx expo install --fix` inside `mobile/`. Shared pieces (`src/lib/api.ts` , components, theme) are agreed between the two of you. The customer screens in the design are still being finished: build navigation, auth and the screens that are final first.

4 Timeline

If the lecturer gives a different CA date, we move the UML weeks to match it.

WEEK	DATES	EVERYONE DOES
1	Nov 5 – 11	Clone the repo, run the server, read AGENTS.md and the design. Group meeting on the shared class diagram.
2	Nov 12 – 18	Use case + class diagram of your module, pushed to `docs/uml/<module>/` .
3	Nov 19 – 25	Sequence + activity diagrams. Lead reviews. **CA UML ready.**
4 – 5	Nov 26 – Dec 9	Server endpoints from `docs/API.md` . Mobile: navigation, auth, theme. Web: layout and dashboard shell.
6 – 7	Dec 10 – 23	Screens on the real API. First end-to-end flow: register parcel → scan → deliver with code.
–	Dec 24 – Jan 3	Holidays: light work only.
8 – 9	Jan 4 – 17	Connect modules: payments to Finance, maintenance blocks trips, licence check, live map.
10	Jan 18 – 24	Test together on real phones, fix bugs, deploy server and web.
11 – 12	Jan 25 – Feb 7	Polish, report, slides, demo script. **Defense rehearsal.**

5 The lecturer's questions

If we cannot answer these at the presentation, there is no A. Every member must be able to explain all six answers, not only the Parcel owner.

How do we solve tracking?	Each parcel gets a QR label and a tracking code. Every handover is scanned and saved as a `ParcelEvent` (who, when, where). The bus shares its GPS during the trip. Sender and receiver get an SMS at each step.
How do we make sure the delivery guy delivers?	The parcel always has one named `currentHolder`. Delivery only counts with the **receiver's 4-digit code** sent by SMS, plus a photo and the GPS location. Late parcels raise an alert for the branch manager.
What is sent is what is received?	At registration the agent records declared contents and value, **takes photos before sealing**, closes the parcel with a **numbered seal** and **weighs** it.
Not tampered with?	On arrival the parcel is **weighed again**: more than 0.1 kg difference puts it ON HOLD. The receiver checks the seal number and the photos **before** giving the code, and can refuse (a claim is opened). Optional insurance pays if something is lost.
How do we earn money?	A % commission on every parcel fee, a booking fee per online ticket, insurance %, home delivery fee, cash-on-delivery fee, and a monthly subscription per agency.
Does the payment reach the right person?	Payments go through MTN MoMo / Orange Money. Cash on delivery is held in **escrow** and released **only** when the receiver's code is correct. Cash collected by staff is checked in the daily reconciliation. Everyone gets an SMS receipt.

6 How to start today

1. Accept the GitHub invite, then clone:

```
git clone https://github.com/fanyicharllson/transitpro.git
```

2. Read `AGENTS.md`, `docs/TEAM_PLAN.md` and `docs/API.md`. If you use an AI agent, tell it: "Read AGENTS.md first and only work in my module."
3. Run the server: `cd server`, `npm install`, copy `.env.example` to `.env` with the database URL the Lead shares privately, then `npm run dev`.
4. Make a branch for each task: `feat/<module>-<what>`. Never push to `main`. Open a Pull Request; the Lead reviews and merges.
5. Weekly check-in in the WhatsApp group every **Sunday evening**: done, next, blocked. Stuck for more than a day? Say it.